# 基于 MPI 的基本矩阵运算

## 1. 实验目的

本次实验的目的是通过大规模矩阵计算，掌握和实践分布式并行编程模型，包括以下核心内容：

a) **掌握 MPI 并行编程：** 深入理解消息传递接口的核心概念与编程范式，Message Passing Interface，MPI 作为实现进程间通信与协作的手段

b) **实现复杂的并行算法：**针对 D = A * B + C 这一大规模矩阵计算任务，设计并实现一个高效、正确的并行算法

c) **掌握关键 MPI 通信操作：**运用 MPI_Bcast 广播操作，以解决所有工作进程均需完整矩阵 B 的计算依赖问题，运用 MPI_Scatterv 非均匀散播与 MPI_Gatherv 非均匀收集操作，以解决矩阵行数 1023 无法被 2 或 4 整除所导致的负载不均与数据非对齐问题

d) **验证并行计算的正确性：** 确保并行计算所得的 result.txt 文件，能通过作业提供的 verify_ 程序与标准答案 RandMatrixAns.txt 进行比对

e) **分析并行性能指标：**通过测量程序在 1 核（串行）、2 核、4 核配置下的总运行时间，包含 I/O 和通信与计算，来量化评估并行化带来的性能提升，计算并对比不同核心数下的并行加速比与并行效率，并分析影响性能的瓶颈

## 2. 并行策略与算法设计

针对 D = A * B + C 这一任务，且在不使用共享内存的情况下，本次实验设计了一个主-从模式并行算法。

### a）核心思想

算法的核心思想计算量最大的 A 矩阵(1023x2047)和 C 矩阵(1023x1023)按行切分，分配给所有可用的 CPU 核心

**进程 0：**扮演主进程 Master 的角色，负责：

1. 串行读取 RandMatrix.txt 中的 A, B, C 三个完整矩阵

2. 启动全局计时器 MPI_Wtime
3. 向所有进程（包括自己）分发计算任务所需的数据
4. 从所有进程收集计算好的结果片段
5. 停止计时器，并将收集拼接好的完整 D 矩阵写入 result.txt

**进程 1...3**: 扮演工作进程(Workers)的角色，负责接收数据、执行本地计算、并返回结果

### b) 数据分发策略

数据分发是本算法的性能关键点，采用了两种不同的 MPI 通信模式

**矩阵 B(N*K)-广播**: 在矩阵乘法 A*B 中，A 的每一行都需要与 B 的每一列相乘，这说明每一个进程，无论它被分配到 A 的哪几行，都必须拥有完整的 B 矩阵，因此，此处使用 MPI_Bcast 指令，由进程 0 将完整的 B 矩阵广播给所有进程

**矩阵 A(M*N)和 C(M*K)-非均匀散播**: 本算法按行划分 A 和 C，由于总行数 M=1023，这个数字无法被核心数 2 或 4 整除

- 如果使用 MPI_Scatter 均匀散播，程序将出错或导致数据错乱，因此使用 MPI_Scatterv 可变长度散播
- 程序首先在所有进程上计算好如何切分 1023 行,例如 4 核心时，切分为 256,256,256,255 行，计算出每个进程应接收的行数和在原始矩阵中的偏移量
- 进程 0 调用 MPI_Scatterv 将 A 和 C 的行块精确分发给对应的进程

### c) 本地并行计算

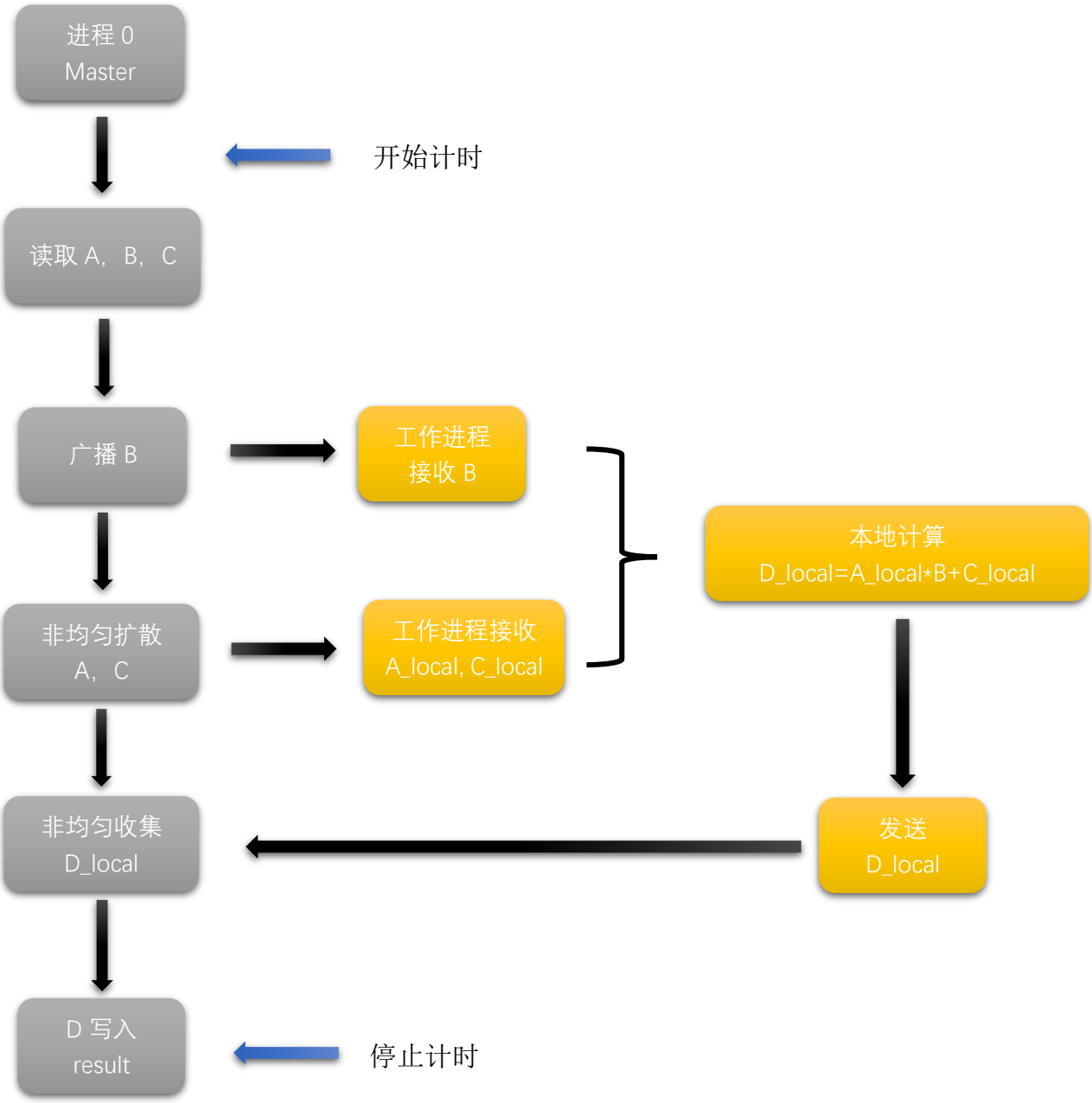
数据分发完毕后，所有进程同时开始执行本地计算，每个进程都执行相同的核心循环，计算 D_local = A_local * B + C_local，这部分是真正实现并行加速的阶段

d）结果收集

**矩阵 D(M*K)-非均匀收集**：与分发同理，由于每个进程计算得到的 D_local 结果块大小不一致（行数不同），因此必须使用 MPI_Gatherv

- 所有工作进程将自己的 D_local 发送给进程 0
- 进程 0 使用与 MPI_Scatterv 相同的行数和偏移量参数，按正确的顺序将这些结果片段拼接回一个完整的 D 矩阵中

e）流程图



## 3. 核心代码分析

本次实验使用 C++语言并基于 MPI 框架实现

### a) 初始化与身份识别

代码的起点是 MPI 环境初始化，并给每个进程标识

```
int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);// 启动 MPI 环境

    int world_rank, world_size;
    // 获取当前进程编号（0-3）
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
    // 获取总进程数（4）
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    Matrix A, B, C, D;
```

World_rank 变量让进程知道自己的身份，world_rank==0 的进程被指定为主进程，其余则为工作进程

### b) 数据分发

主进程负责数据读取与分发逻辑

```
if (world_rank == 0) {
    // 1. 只有 0 号进程读取文件
    start_time = MPI_Wtime(); // 启动计时
    readMatrices("RandMatrix.txt", A, B, C);
}
// ... 计算 Scatterv 所需的 sendcounts 和 displs 数组 ...
// (这一步计算如何将 1023 行分配给 world_size 个进程)

// 2. 广播 B 矩阵
// 0 号进程作为 root（发送方），其他进程 (1, 2, 3) 作为接收方
// 所有人最终都拥有完整的 B
MPI_Bcast(B.data(), N * K, MPI_LONG_LONG, 0, MPI_COMM_WORLD);

// 3. 非均匀散播 A 和 C 矩阵
// 0 号进程(root)将 A 切片，分发给所有人（包括自己）
MPI_Scatterv(
    world_rank == 0 ? A.data() : nullptr, // 只有 0 号有数据
    sendcounts_A.data(),
    displs_A.data(),
    MPI_LONG_LONG,
```

```
    A_local.data(), // 每个进程的接收缓冲区
    sendcounts_A[world_rank], // 每个进程接收的数量
    MPI_LONG_LONG,
    0, MPI_COMM_WORLD
);
// MPI_Scatterv(C.data(), ...) 同理
```

MPI_Bcast 是一对多的广播，所有进程在计算时都需要完整的 B

MPI_Scatterv 是一对多的散播，允许不均匀散播

### c）并行计算

数据分发完成后，所有进程同时计算，处理各自的 A_local,C_local

```
 for (int i = 0; i < local_rows; ++i) {
        for (int j = 0; j < K; ++j) {
            long long sum = 0;
            for (int k = 0; k < N; ++k) {
                // A_local[i][k] * B[k][j]
                sum += A_local[i * N + k] * B[k * K + j];
            }
            // D_local[i][j] = sum + C_local[i][j]
            D_local[i * K + j] = sum + C_local[i * K + j];
        }
    }
```

该部分是程序中并行加速的地方，也是最耗时的部分

### d）结果收集

计算完成后，主进程将各处的 D_local 收集拼成完整的 D

```
// 5. 非均匀收集 D_local
// 所有进程（包括 0）都是发送方，
// 只有 0 号进程是 root（接收方）
MPI_Gatherv(
    D_local.data(),             // 发送本地 D_local
    sendcounts_C[world_rank],   // 发送的数量（与 C 相同）
    MPI_LONG_LONG,
    world_rank == 0 ? D.data() : nullptr, // 只有 0 号接收
    sendcounts_C.data(), // 0 号按顺序接收
    displs_C.data(),     // 0 号按偏移量存放
    MPI_LONG_LONG,
    0, MPI_COMM_WORLD
);
```

MPI_Gatherv 是 MPI_Scatterv 的逆操作，根据参数收集数据块，并正确拼装

## 4. 实验结果分析

### a) 正确性检验

```
wyj2350489@ps:~/MPI-project$ ./verify_ RandMatrix.txt result.txt
比对成功!
```

### b) 性能测试数据

包含 I/O 的计时，在 1 核（串行）、2 核、4 核的环境下分别运行

| 核心数（p） | 运行时间（s） |
|---|---|
| 1 | 5.400121 |
| 2 | 4.135167 |
| 4 | 3.033115 |

### c) 性能指标

两个指标：加速比与效率

**加速比（S_p）= T_serial / T_parallel(p)（串行/并行）**

**效率（E_p）= S_p / p**

| 核心数（p） | 运行时间（s） | 加速比（S_p） | 效率（E_p） |
|---|---|---|---|
| 1 | 5.400121 | 1.000 | 100.0% |
| 2 | 4.135167 | 1.306 | 65.3% |
| 4 | 3.033115 | 1.780 | 44.5% |

### d) 结果分析

**1. 加速比分析：**

程序获得了有效的加速比，4 核（1.780 倍）比 2 核（1.306 倍）快，两者都显著快于 1 核（串行）。但加速未达到先行加速，加速比理论上要接近核数，说明存在额外开销

**2. 效率分析：**

并行效率随核心数增加而急剧下降，从 2 核的 65.3% 降低到 4 核的

44.5%，这表明，随着使用更多核心加深并行化程度，“开销”所占总时间的比例正在迅速增大，抵消了并行计算带来的一部分收益

**3．瓶颈定位：**

效率低下的核心原因符合阿姆达尔定律，即程序的总加速比受限于其串行部分，在程序中，该瓶颈主要来自以下几点

- **I/O 串行开销（主要瓶颈）：** readMatrices 读取 A, B, C 和 writeResult 写入 D 的任务只能由 0 号进程串行完成，从数据上看，1 核总时间为 5.40 秒，而 4 核总时间仍需 3.03 秒，这说明串行 I/O 占用了总运行时间中的巨大部分，这部分时间是无法通过增加核心数来并行的，它为总加速比设定了一个很低的上限
- **通信开销：** MPI_Bcast 需要将完整的 B 矩阵广播给所有进程，MPI_Gatherv 需要从所有进程收集结果，这些通信操作本身需要时间，并且随着核心数增加，网络争用和同步等待时间会更长
- **负载不均衡：** 由于 M=1023，无法被 2 或 4 整除，MPI_Scatterv 会导致某些进程多计算 1 行，这会导致先算完的进程必须等待还在计算的进程，造成时间浪费，降低了效率

## 5. 总结

本次实验成功地使用 C++ 和 MPI 消息传递接口框架，在分布式系统环境下，实现了大规模矩阵计算 D = A*B + C

a) **正确性：**通过 verify_ 程序验证，本算法在 1、2、4 核心配置下均能正确完成计算
b) **并行有效性：**实验数据表明，使用 4 核（1.780 倍加速比）和 2 核（1.306 倍加速比）均比 1 核心（串行）运行更快，证明了并行策略是有效的
c) **性能瓶颈：**实验的核心发现是，程序的并行效率随核心数增加而急剧下降（4 核心时仅为 44.5%），根据阿姆达尔定律，这暴露出程序的总加速比受到了串行部分的严重限制
d) **瓶颈分析：**最大的瓶颈在于串行 I/O，只有 0 号进程负责读写文件，这部

分耗时在总共 5.4 秒中占了很大比例，且无法通过增加 CPU 核心来缩短，从而为整体性能设定了很低的上限

e) **总结**：本实验验证了按行划分的 MPI 算法的可行性，但也证明了串行 I/O 是分布式计算中必须优先解决的性能瓶颈